

Pipeline de dados e previsão de churn

Do dado bruto ao modelo de IA: quatro fontes, quatro camadas no PostgreSQL e um modelo servido por API

Sandro

Fundamentos de Banco de Dados (BDED-2026.1)

Prof. Dr. Adolfo Colares · UNIFAP Digital

[github.com/xsansil/
pipeline-churn-ecommerce](https://github.com/xsansil/pipeline-churn-ecommerce)

Quais clientes estão prestes a parar de comprar?

Reter custa pouco

Um cupom de desconto para quem talvez nem fosse embora.

Perder custa muito

Todo o faturamento futuro daquele cliente.

A restrição

A equipe não aborda 400 pessoas. Precisa de ordem.

Definição adotada: deu churn o cliente que não fez nenhuma compra entre 1º de outubro e 31 de dezembro de 2024.

O modelo não adivinha. Ele ordena.

100% dos clientes no decil de maior risco deram churn

9% no decil de menor risco

Decil	Geral	Só teste
1 (maior risco)	100,0%	100,0%
2	87,5%	77,8%
5	65,0%	60,0%
9	22,5%	35,7%
10 (menor risco)	5,1%	9,1%

A coluna “**só teste**” é a que vale. A geral inclui os 299 clientes que o modelo viu no treino, e sai otimista.

Quatro fontes heterogêneas

#	Tipo	Conteúdo	Registros	Por que existe
1	CSV	extrato de transações	6.383	o fato central
2	JSON	catálogo de produtos	12	traz o custo, e daí a margem
3	API REST	municípios do IBGE	237	mesorregião, que o extrato não tem
4	PostgreSQL	cadastro do CRM	388	fonte autoritativa do cliente

A única externa de verdade

A API do IBGE sai do ar. A coleta tem cache em disco e degrada por UF: se um estado falha, os outros entram e o cache cobre o buraco.

Uma lacuna deliberada

O CRM não tem 12 dos 400 compradores. Sem isso o join casaria 100% e não haveria decisão a tomar.

De 6.383 linhas sujas a 5.809 transações

Etapa	Entrada	Saída	Removidos
1. Duplicatas exatas	6.383	6.198	185
3. Data ausente ou impossível	6.198	6.106	92
4. Valor ausente	6.106	5.986	120
5a. Valor menor ou igual a zero	5.986	5.929	57
5b. Quantidade fora de 1 a 20	5.929	5.881	48
6. Preço unitário implausível	5.881	5.809	72

O IQR erraria nas duas pontas

O intervalo interquartil removeria **587** registros, liderados por monitor, SSD e headset: os produtos caros e legítimos.

O critério adotado compara o **preço unitário** com o teto do catálogo. Isolou os **72** erros de digitação sem eliminar venda boa.

Sete etapas, cada uma contando entrada, saída e motivo. Descarte total de 9,0%.

O que a integração recuperou

Indicador	Fonte única	Integrado
Idades imputadas pela mediana	296	4
Cidades marcadas “Desconhecida”	170	1
E-mails inválidos	43	1
Transações com mesorregião	zero	5.808
Margem por categoria	não calculável	22% a 55%

O que a limpeza de fonte única tinha que **adivinhar**, o cadastro do CRM **forneceu**.

Quatro camadas, quatro papéis

bronze

O dado como chegou

Tudo TEXT, sem restrição, sem descarte. Aceita lixo de propósito.

silver

Limpo e enriquecido

Tipos corretos, chave primária, CHECK e índices. Três junções.

gold

Pronto para o modelo

Uma linha por cliente, 44 features e o alvo. Sete views de consumo.

meta

Proveniência

Execuções, marcos de carga incremental e histórico de treinos.

Cada camada é um *schema*. Quem consome o gold não precisa de acesso ao bronze, e a permissão pode ser diferente por camada.

Todo join tem contador

`merge(how="left")` nunca levanta erro. Quando a chave não casa, ele preenche nulo e segue.

Um join que deveria casar 100% e casa 3% produz uma tabela do tamanho certo, com as colunas certas, cheia de nulos. O problema só aparece lá na frente, como “o modelo não aprende”.

Junção	Chave	Resultado registrado
Cadastro do CRM	cliente_id	5.674 casaram, 135 de 12 clientes sem cadastro
Catálogo (custo)	produto sem acento	5.809 casaram, zero sem custo
IBGE (mesorregião)	cidade sem acento + UF	5.808 casaram, 1 sem região

A junção com o catálogo **levanta exceção** se sobrar produto sem custo: sem ele a margem sai errada em silêncio.

44 features, e uma linha no tempo

O erro que parece acerto

Definir churn como “90 dias sem comprar” e deixar essa variável entre as features põe a resposta dentro da pergunta. O modelo acerta quase tudo sem ter aprendido nada.

A separação temporal

A pergunta usa dados de **antes** do corte; a resposta vem do que aconteceu **depois**. O modelo nunca vê o futuro.

observação

2023-01-01 a 2024-09-30

5.416 transações constroem as features

avaliação

2024-10-01 a 2024-12-31

393 transações definem o alvo

Maior correlação de uma feature com o alvo: **0,506**. Acima de 0,90 seria indício de vazamento, e um teste barra isso.

Cinco algoritmos comparados

Modelo	ROC AUC	Validação cruzada	F1	Acurácia
Regressão Logística (escolhido)	0,731	$0,818 \pm 0,042$	0,684	0,66
Random Forest	0,714	$0,829 \pm 0,048$	0,732	0,70
KNN (k=15)	0,697	$0,826 \pm 0,047$	0,739	0,71
Gradient Boosting	0,694	$0,811 \pm 0,036$	0,713	0,69
Árvore de Decisão	0,647	$0,700 \pm 0,041$	0,636	0,62
Linha de base (sempre churn)	0,500	n/a	0,718	0,56

Olhe a última linha. A linha de base não aprendeu nada, e o F1 dela supera o de **quatro dos cinco** modelos treinados.

Selecionar por F1 escolheria o modelo errado

Quem responde “churn” para todo mundo acerta **todos** os que deram churn, então o recall é 1,0. E como 56% dos clientes de fato deram churn, a precisão já nasce em 0,56.

A média harmônica disso é alta. O F1 só enxerga a classe positiva, e nunca olha para os verdadeiros negativos.

Por F1, o vencedor seria o KNN

F1 = 0,718

da linha de base que não aprendeu nada

Em ROC AUC ela marca

0,500

O que a integração *não* entregou

Origem	Features	Maior correlação	Relevantes
extrato CSV	33	0,506	14
catálogo JSON	3	0,379	1
API IBGE	6	0,156	0
CRM PostgreSQL	1	0,023	0
junção CSV × CRM	1	0,007	0

A única feature forte vinda da integração, **margem_total**, correlaciona **0,969 com total_gasto**. É o mesmo dado reescalado, não informação nova.

A integração melhorou muito a **qualidade** do dado e quase nada o **poder preditivo**. São coisas diferentes.

O pipeline funcionando

```
$ python pipeline.py
```

```
ETAPA 0: PREPARAR O BANCO
```

```
ETAPA 1: COLETA DAS FONTES
```

```
ETAPA 2: LIMPEZA E INTEGRAÇÃO
```

```
ETAPA 3: ENGENHARIA DE FEATURES
```

```
ETAPA 4: TREINO E SELEÇÃO
```

```
CONCLUÍDO em 13.9 segundos
```

A carteira ordenada

```
GET /clientes/risco
```

Previsão para um cliente

```
POST /prever
```

O histórico de treinos

```
SELECT * FROM meta.treinos
```

Um clone limpo, sem dados e sem modelo, reproduz tudo do zero: **6.383 → 5.809 → 399 clientes → ROC AUC 0,731.**

O QUE FICA

4

fontes heterogêneas integradas

44

features documentadas, com a origem de cada uma

46

testes automatizados

14s

do CSV bruto ao modelo treinado

As três lições que o projeto deixou

Toda falha silenciosa precisa de um contador. O join que não casa, a coluna que perde a restrição, o scaler esquecido: nada disso levanta erro.

Métrica sem linha de base não significa nada. F1 de 0,718 parecia bom até a linha de base tirar o mesmo.

Resultado negativo também é resultado. A integração não melhorou a previsão, e isso está escrito na entrega.

github.com/xsansil/pipeline-churn-ecommerce